

PvrGPU Geometry Pipeline — Block Diagram, Data Path & Pseudocode

本文件以 block diagram、data path flow 及 pseudocode 形式詳述 `src/systemc/geometry/` 五個 SystemC module 的行為。
這五個 module 以 bounded FIFO 串聯，依序將 CPU 提交的 draw call 轉換為可供 fragment pipeline 消費的 tile-based 參數。

0. Top-Level Block Diagram

Geometry Pipeline — Top-Level Block Diagram

5 個 SC_THREAD module 以 bounded sc_fifo 串聯
FIFO 只傳 PipelineTxn handle；大型 payload 全存 MemoryPool

① VDM (Vertex Data Master)

IN: DrawCommand, VBO descriptors, IndexBuffer
OUT: validated IA counters
STAGE: kSubmitted -> kVdmComplete

↓ sc_fifo<PipelineTxn>

② VertexFetch

IN: VBO raw bytes, index buffer
OUT: VertexLane[] (VTXIN regs), VertexLaneRef[]
STAGE: kVdmComplete -> kVertexFetched

↓ sc_fifo<PipelineTxn>

[Vertex Shader — USC / PDS / PCO ISS]

IN: VertexLane[].vertex_input
OUT: VertexLane[].vertex_output (VTXOUT 0..63)
STAGE: kVertexFetched -> kVertexShaded

↓ sc_fifo<PipelineTxn>

③ ClipCull

IN: shaded VertexLane[], VertexLaneRef[], indices
OUT: RasterTriangle[], raster_vertex_outputs[]
STAGE: kVertexShaded -> kClipCullComplete

↓ sc_fifo<PipelineTxn>

④ Tiler

IN: RasterTriangle[]
OUT: TileRecord[], TilePrimitiveRef[]
STAGE: kClipCullComplete -> kTiled

↓ sc_fifo<PipelineTxn>

⑤ ParameterBuffer

IN: RasterTriangle[], raster_vertex_outputs[]
OUT: ParameterTriangle[], ParameterCoefficientSet[]
STAGE: kTiled -> kParameterBufferReady

↓ sc_fifo<PipelineTxn>

[Fragment Pipeline: TileScheduler -> ISP -> USC -> PBE]

1. Data Path Flow — 完整資料流向圖

以下展示每個 module 讀取 (R) 與寫入 (W) 的 MemoryPool payload handle :

CPU Submit

W: PipelineState (DrawCommand, RasterState, functional_case)
W: vertex_buffer_resources[], vertex_attribute_bindings[]
W: vertex_indices[] (index buffer, 若 indexed draw)
W: drawlist_stats[]
W: vertex_code, vertex_instructions, fragment_code (shader binary)

① VDM

R: drawlist_stats -> 驗證 DrawList 0
R: vertex_buffer_resources -> VBO descriptor 驗證
R: vertex_attribute_bindings -> binding layout 驗證
R: vertex_indices -> index buffer 解碼 (若 DRAM: 發 memory read)
W: state.counters.ia_vertices, ia_primitives, drawlists
W: state.vertex_indices (若從 DRAM 讀入後存入 pool)

② VertexFetch

R: vertex_buffer_resources -> VBO descriptors
R: vertex_attribute_bindings -> binding layout
R: VBO raw data (pool handle 或 DRAM gpu_address)
R: vertex_indices -> 解碼後的 index array
W: vertex_lanes[] -> VertexLane (每個 unique vertex 的 VTXIN regs)
W: vertex_lane_refs[] -> VertexLaneRef (每個 index occurrence 映射)
W: state.counters.vs_invocations, vertex_attribute_fetches/bytes

[Vertex Shader]

R: vertex_lanes[].vertex_input (VTXIN register bank)
W: vertex_lanes[].vertex_output (VTXOUT 0..63, 含 clip position)
W: vertex_lanes[].emitted, .ended flags

③ ClipCull

R: vertex_lanes[] -> shaded VTXOUT (clip position + varyings)
R: vertex_lane_refs[] -> index-to-lane mapping
R: vertex_indices[] -> original indices (indexed path)
R: shader_varying_bindings[] -> exact VS-FS linkage
W: raster_triangles[] -> clipped/culled RasterTriangle
W: raster_vertex_outputs[] -> 攤平 per-vertex VTXOUT (uint32[])
W: state.counters.c_invocations, c_primitives

④ Tiler

R: raster_triangles[] -> bounding box + rasterizable flag
W: tile_records[] -> TileRecord (32x32 grid, row-major)
W: tile_primitive_refs[] -> TilePrimitiveRef (per-tile prim list)
W: state.counters.setup_triangles, tiles_binned

⑤ ParameterBuffer

R: raster_triangles[] -> screen coords, reciprocal_w, VTXOUT stride
R: raster_vertex_outputs[] -> per-vertex varying values
R: shader_varying_bindings[] -> interpolation mode per varying
W: parameter_triangles[] -> edge equations + bbox + signed_area
W: parameter_coefficients[] -> A/B/C plane coefficients
W: (或 DRAM write 到 0x30000000 / 0x34000000)
W: state.counters.parameter_coefficient_sets, parameter_write_bytes

[Fragment Pipeline]

R: tile_records[] -> 排程哪些 tile 要光柵化
R: tile_primitive_refs[] -> 每個 tile 裡有哪些 primitives
R: parameter_triangles[] -> edge test + depth interpolation
R: parameter_coefficients[] -> varying interpolation

2. VDM Internal Block Diagram

VDM — Internal Processing Flow

Step 1: Stage Guard

ASSERT state.stage == kSubmitted
ASSERT is_raster_case(functional_case)
validate memory_mode consistency

Step 2: Validate DrawList

Load drawlist_stats[]
ASSERT exactly DrawList 0

Step 3: Validate Vertex Input State

Load vertex_buffer_resources[] + bindings[]
Check VBO payloads exist and don't alias
Compute vertex_capacity from stride/offset/VBO size

Step 4a: Direct Draw Path

IF FillSolid/Texture/DriverPcoTriangles:
ASSERT topology == TriangleStrip(4v)/List(6v/Nv)
ASSERT no index buffer present
ia_vertices = vertex_count
ia_primitives = vertex_count/3 or 2

Step 4b: Indexed Draw Path

ELSE (indexed triangle raster):
Load/decode index buffer (uint8/16/32)
IF DRAM: gpu_memory.read -> pool.store
Split by primitive_restart_index
Count ia_vertices (valid index occurrences)
Count ia_primitives per topology per segment

Step 5: Finalize

state.stage = kVdmComplete
cycles = base + ceil(ia_vertices / vertices_per_batch)
WaitForCycles(cycles); store; output.write

3. VertexFetch Internal Block Diagram

VertexFetch — Internal Processing Flow

Step 1: Load Vertex Input State

Load VBO descriptors + attribute bindings
Load raw VBO bytes (pool or DRAM read)
Validate VTXIN destination register non-overlap
Accumulate bytes_per_vertex

Step 2a: Direct Path (non-indexed)

FOR vertex = 0..vertex_count-1:
lanes.append(MakeLane(vertex, vertex_input))
IF driver_pco: lane_refs.append({lane_idx, vtx})

Step 2b: Indexed Path — Topology Expansion

```
IF topology != TriangleList OR has_restart OR !u16:  
expanded = ExpandTopology(indices, topology)  
TriangleStrip: even{i,i+1,i+2} odd{i+1,i+2}  
TriangleFan: {v0, vi, vi+1}  
Lines/LineStrip/Loop: degenerate tris  
Points: {vi, vi, vi}  
Rewrite vertex_indices as uint16 TriangleList
```

Step 2c: Indexed Path — Segmented Cache Fetch

```
WHILE occurrence < index_count:  
segment = min(segment_max, remaining)  
cache = CacheEntry[post_transform_cache_slots]  
FOR EACH index IN segment:  
vertex_index = expanded[i] + base_vertex  
entry = cache[vertex_index % slots]  
IF MISS: lanes.append(MakeLane(vertex_index))  
lane_refs.append({entry.lane_index, vertex_index})
```

Step 3: MakeLane — Attribute Fetch Core

```
FOR EACH binding:  
offset = vertex_index * stride + binding.offset  
FOR comp = 0..dst_components-1:  
IF comp < src_components:  
read from VBO (float/half/int/normalized)  
ELSE:  
default = (comp==3) ? 1.0f : 0.0f  
lane.vtxin[dst_reg+comp] = float_bits(value)
```

Step 4: Finalize

```
state.vertex_lanes = store(lanes)  
state.vertex_lane_refs = store(lane_refs)  
counters: vs_invocations, attribute_fetches/bytes  
state.stage = kVertexFetched  
cycles = base + ceil(attr_bytes / bytes_per_batch)
```

4. ClipCull Internal Block Diagram

ClipCull — Internal Processing Flow

Step 1: Load Shaded Data

```
Load vertex_lanes[] (已含 VTXOUT 0..63)  
Load vertex_lane_refs[] (indexed path)  
Load vertex_indices[] (indexed path)  
Load shader_varying_bindings[] (if has varyings)  
Determine active_vertex_output_dwords
```

Step 2a: Direct Path (FillSolid/Texture)

```
4 or 6 lanes -> 2 triangles  
Verify all vertices inside homogeneous clip space  
No clipping needed (guaranteed inside)  
Strip indexing: {0,1,2},{2,1,3}  
List indexing: {0,1,2},{3,4,5}
```

Step 2b: Indexed Path — Segment ClipMask Check

```
FOR EACH segment (matches VertexFetch segments):
generic_clip_path = false
FOR EACH vertex IN segment:
mask = ClipMask(vertex)
// 6 frustum planes + 8 clip distance planes
// PlaneDistance: right(-x+w), left(x+w),
// top(-y+w), bottom(y+w), near(z+w), far(-z+w)
IF mask != 0: generic_clip_path = true
```

Step 3: Sutherland-Hodgman Clipping

```
FOR EACH triangle (3 consecutive lane_refs):
Read 3 ClipVertex from shaded lanes
masks[3] = ClipMask for each vertex
IF all_inside (union==0): polygon = original 3v
IF all_outside_same (intersection!=0): polygon={}
ELSE: clip against each active plane:
FOR EACH edge of polygon:
prev_dist = PlaneDistance(prev, plane)
curr_dist = PlaneDistance(curr, plane)
IF prev inside: emit prev
IF sign_change: emit Interpolate(t, out, in)
t = dist / (dist - prev_dist)
Interpolate ALL VTXOUT components linearly
```

Step 4: Fan Emission + Face Cull

```
FOR fan_idx = 2 .. polygon.size()-1:
fan_tri = {poly[0], poly[fan-1], poly[fan]}
front = ClassifyFrontFacing(fan_tri, winding)
// area = cross_product(ndc) -> CW/CCW
culled = IsFaceCulled(cull_state, front)
// Front/Back/FrontAndBack modes
IF generic_clip_path AND culled: skip entirely
```

Step 5: Viewport Transform + BuildRasterTriangle

```
FOR EACH vertex in fan triangle:
1/w = reciprocal_w = 1.0 / clip_w
ndc_xyz = clip_xyz * reciprocal_w
IF depth_clamp: ndc_z = clamp(ndc_z, -1, 1)
x_win = (ndc_x*0.5+0.5) * width
y_win = (ndc_y*0.5+0.5) * height
z_win = ndc_z*0.5 + 0.5
Quantized area (24.8 fixed-point cross product)
IF area < 0: swap v1/v2 (normalize to CCW)
Serialize per-vertex VTXOUT to raster_vertex_outputs
```

Step 6: Finalize

```
state.raster_triangles = store(triangles)
state.raster_vertex_outputs = store(vtxout_flat)
counters: c_invocations, c_primitives
state.stage = kClipCullComplete
cycles = base + ceil(c_invocations / prims_per_batch)
```

5. Tiler Internal Block Diagram

Tiler — Internal Processing Flow

Step 1: Load + Validate

```
Load raster_triangles[] from pool
ASSERT count == c_primitives
Compute tile grid: tiles_x = ceil(width/32)
tiles_y = ceil(height/32)
```

Step 2: Row-Major Tile Scan + Bounding-Box Binning

```
FOR y0 = 0, step 32, y0 < height:
FOR x0 = 0, step 32, x0 < width:
tile = {x0, y0, min(x0+32,W), min(y0+32,H)}
tile.first_primitive_ref = refs.size()
FOR EACH triangle[i]:
IF !rasterizable: SKIP
tri_bbox = {min/max of x[0..2], y[0..2]}
IF tri_bbox overlaps tile:
refs.append({param_idx:i, submit_ordinal})
tile.primitive_ref_count = new_refs_count
```

Step 3: Finalize

```
state.tile_records = store(tiles)
state.tile_primitive_refs = store(refs)
counters: setup_triangles, tiles_binned
state.stage = kTiled
cycles = base + ceil(setup_triangles / tri_per_batch)
```

6. ParameterBuffer Internal Block Diagram

ParameterBuffer — Internal Processing Flow

Step 1: Load Input Data

```
Load raster_triangles[] from pool
Load raster_vertex_outputs[] (flat VTXOUT array)
Load shader_varying_bindings[] (if has varyings)
```

Step 2: Non-Rasterizable Placeholder

```
IF !triangle.rasterizable:
Emit ParameterTriangle with identity only
No edge equations, no coefficients
Preserves parameter_index for identity tracking
CONTINUE to next triangle
```

Step 3: Edge Equations (24.8 fixed-point)

```
FOR edge IN {0, 1, 2}:
next = (edge+1) % 3
qx0 = round(x[edge] * 256) // Quantize
qy0 = round(y[edge] * 256)
qx1 = round(x[next] * 256)
qy1 = round(y[next] * 256)
eq.a = qy0 - qy1
eq.b = qx1 - qx0
eq.c = qx0*qy1 - qx1*qy0
// Top-left fill rule:
dy = qy1 - qy0; dx = qx1 - qx0
eq.inclusive = (dy<0) OR (dy==0 AND dx>0)
```

Step 4: Signed Area + Bounding Box

```
signed_area = (x1-x0)*(y2-y0) - (y1-y0)*(x2-x0)
(quantized 24.8 cross product)
ASSERT signed_area > 0 (CCW winding guaranteed)
bbox.min_x = clamp(floor(min(x[])), 0, width)
bbox.min_y = clamp(floor(min(y[])), 0, height)
bbox.max_x = clamp(ceil(max(x[])), 0, width)
bbox.max_y = clamp(ceil(max(y[])), 0, height)
```

Step 5: Varying Coefficient Sets (if applicable)

```
5a. 1/W plane: BuildPlane(triangle, reciprocal_w[3])
det = (x1-x0)*(y2-y0) - (x2-x0)*(y1-y0)
a = ((v1-v0)*(y2-y0)-(v2-v0)*(y1-y0)) / det
b = ((x1-x0)*(v2-v0)-(x2-x0)*(v1-v0)) / det
c = v0 - a*x0 - b*y0
5b. Per-varying-component coefficient:
FOR EACH binding, FOR EACH component:
IF FLAT: coeff = {0, 0, provoking_vertex_v}
IF NOPERSP: coeff = BuildPlane(tri, varying[])
IF SMOOTH: coeff = BuildPlane(tri, v*1/w[])
```

Step 6: Output + Finalize

```
IF DRAM mode:
gpu_memory.write(0x30000000, parameter_triangles)
gpu_memory.write(0x34000000, coefficients)
ELSE:
state.parameter_triangles = pool.store(params)
state.parameter_coefficients = pool.store(coeffs)
state.stage = kParameterBufferReady
cycles = base + ceil(setup_triangles / tri_per_batch)
```

7. Key Data Structures (Block Diagram)

7.1 Per-Stage MemoryPool Handle Map

Stage	Handle 名稱	資料型別	生產者	消費者
Submit	vertex_buffer_resources	VertexBufferResource[]	CPU	VDM, VertexFetch
Submit	vertex_attribute_bindings	VertexAttributeBinding[]	CPU	VDM, VertexFetch
Submit	vertex_indices	uint8/16/32[]	CPU/VDM	VDM, VertexFetch, ClipCull
Submit	drawlist_stats	DrawListStats[]	CPU	VDM
VertexFetch	vertex_lanes	VertexLane[]	VertexFetch	Shader, ClipCull
VertexFetch	vertex_lane_refs	VertexLaneRef[]	VertexFetch	ClipCull
ClipCull	raster_triangles	RasterTriangle[]	ClipCull	Tiler, ParameterBuffer
ClipCull	raster_vertex_outputs	uint32[]	ClipCull	ParameterBuffer
Tiler	tile_records	TileRecord[]	Tiler	Fragment
Tiler	tile_primitive_refs	TilePrimitiveRef[]	Tiler	Fragment
ParamBuf	parameter_triangles	ParameterTriangle[]	ParameterBuffer	Fragment

ParamBuf	<code>parameter_coefficients</code>	<code>ParameterCoefficientSet[]</code>	ParameterBuffer	Fragment
----------	-------------------------------------	--	-----------------	----------

7.2 RasterTriangle 結構

RasterTriangle

`x[3], y[3]` : window-space 座標 (float)
`window_z[3]` : window-space depth
`reciprocal_w[3]` : $1/\text{clip_w}$ (for perspective)
`front_facing` : 0 or 1
`rasterizable` : 0 or 1
`face_culled` : 0 or 1
`key.submit_ordinal`: 提交順序
`key.api_primitive_id`: API primitive 索引
`key.clip_piece` : clipping fan piece index
`first_vertex_output_dword`: VTXOUT 起始偏移
`vertex_output_stride_dwords`: 每頂點 VTXOUT 數

7.3 ParameterTriangle 結構

ParameterTriangle

`edge[3]`: EdgeEquation {a, b, c, inclusive}
`a` = Δy (24.8 fixed-point)
`b` = $-\Delta x$
`c` = $x_0*y_1 - x_1*y_0$
`inclusive` = top-left rule flag
`signed_area`: 24.8 cross product (>0 for CCW)
`min_x, min_y, max_x, max_y`: pixel bbox
`window_z[3]`: per-vertex depth
`first_coefficient_set`: coefficient array offset
`coefficient_set_count`: number of coeff sets
`key`: same as RasterTriangle.key
`front_facing, rasterizable, face_culled`

7.4 Interpolation Coefficient 結構

ParameterCoefficientSet

`a`: dF/dx (float32 as uint32 bits)
`b`: dF/dy (float32 as uint32 bits)
`c`: $F(0,0)$ (float32 as uint32 bits)
`pad`: reserved (0)
 用途: $f(x,y) = a*x + b*y + c$
 Smooth varying: 存 v/w 的 plane，搭配 $1/w$ plane
 Flat varying: $a=0, b=0, c=\text{provoking value}$
 NoPerspective: 直接存 v 的 plane

8. Module 共通模式

所有 geometry module 遵循相同的 SystemC event-driven 模式：

模式	說明
<code>SC_THREAD(Run)</code>	建構時註冊為 SystemC thread，無限迴圈
Bounded FIFO	<code>sc_fifo_in<PipelineTxn></code> / <code>sc_fifo_out<PipelineTxn></code>
MemoryPool handle	FIFO 只傳 handle，大型 payload 由 handle 間接引用

Stage guard	每個 module 要求精確的前置 stage
Fail-closed 驗證	任何不合法的輸入直接 throw，不做 fallback
Reference uArch cycles	<code>WaitForCycles(base + ceil(work / throughput))</code>
Counter 累加	每個 module 累加自己的 counters 到 <code>state.counters</code>



9. 參考 uArch Cycle Model 參數

參數	Module	說明
<code>vdm_base_cycles</code>	VDM	固定啟動成本
<code>vdm_vertices_per_batch</code>	VDM	每 batch 處理的頂點數
<code>vertex_fetch_base_cycles</code>	VertexFetch	固定啟動成本
<code>vertex_fetch_bytes_per_batch</code>	VertexFetch	每 batch 擷取的位元組數
<code>index_segment_max_indices</code>	VertexFetch / ClipCull	分段大小 (必須是 3 的倍數)
<code>post_transform_cache_slots</code>	VertexFetch	Post-transform cache 大小
<code>clip_base_cycles</code>	ClipCull	固定啟動成本
<code>clip_primitives_per_batch</code>	ClipCull	每 batch 裁剪的 primitive 數
<code>tiler_base_cycles</code>	Tiler	固定啟動成本
<code>tiler_triangles_per_batch</code>	Tiler	每 batch 分箱的三角形數
<code>parameter_base_cycles</code>	ParameterBuffer	固定啟動成本
<code>parameter_triangles_per_batch</code>	ParameterBuffer	每 batch 建立參數的三角形數

10. VDM Pseudocode

```
FUNCTION VDM::Run():
  LOOP forever:
    txn = input_fifo.read()
    state = load(pool, txn.handle)
    ASSERT state.stage == kSubmitted

    // 1. Functional case 分派
    ASSERT is_raster_case(state.functional_case)
    validate_drawlist(pool, state.drawlist_stats)

    // 2. 驗證 Vertex Input Layout
    resources[] = load(pool, state.vertex_buffer_resources)
    bindings[] = load(pool, state.vertex_attribute_bindings)
    FOR EACH binding:
      element_bytes = binding.src_components * component_type_size
      ASSERT binding.stride >= element_bytes
      IF binding.instance_divisor != 0: SKIP
      capacity = 1 + (vbo.byte_size - first_end) / stride
      vertex_capacity = min(vertex_capacity, capacity)

    // 3. Direct (non-indexed) 路徑
    IF is_fill_solid / texture / driver_pco_triangles:
      ASSERT topology == TriangleStrip(4v) or TriangleList(6v/Nv)
      iq_vertices = vertex_count
      iq_primitives = vertex_count / 3 (or 2)
```

```

// 4. Indexed 路徑
ELSE:
    IF memory_backed:
        raw = gpu_memory.read(index_buffer_addr, index_buffer_bytes)
        state.vertex_indices = pool.store(raw)
    indices[] = decode_as(uint8/16/32, state.vertex_indices)
    segments = split_by_restart(indices)
    FOR EACH segment: count ia_vertices, ia_primitives by topology

// 5. 完成
state.stage = kVdmComplete
cycles = base + ceil(ia_vertices / vertices_per_batch)
wait(cycles); store(state); output.write(txn)

```

11. VertexFetch Pseudocode

```

FUNCTION VertexFetch::Run():
    LOOP forever:
        txn = input_fifo.read()
        state = load(pool, txn.handle)
        ASSERT state.stage == kVdmComplete

        vertex_input = LoadVertexInputState(pool, state, memory)
        lanes = [], lane_refs = []

        // Direct path
        IF fill_solid / texture / driver_pco:
            FOR vertex = 0..count-1:
                lanes.append(MakeLane(vertex, vertex_input))

        // Indexed path
        ELSE:
            // Expand topology -> TriangleList if needed
            IF needs_expansion:
                expanded = ExpandTopology(indices, topology, restart)
                state.vertex_indices = pool.store(expanded as uint16[])

            // Segmented fetch with post-transform cache
            WHILE occurrence < index_count:
                segment = min(segment_max, remaining)
                cache = CacheEntry[cache_slots]
                FOR EACH index:
                    entry = cache[vertex_index % slots]
                    IF MISS: lanes.append(MakeLane(vtx_idx, input))
                    lane_refs.append({entry.lane_index, vertex_index})

        state.vertex_lanes = store(lanes)
        state.vertex_lane_refs = store(lane_refs)
        state.stage = kVertexFetched
        cycles = base + ceil(attr_bytes / bytes_per_batch)
        wait(cycles); store(state); output.write(txn)

```

12. ClipCull Pseudocode

```

FUNCTION ClipCull::Run():
    LOOP forever:
        txn = input_fifo.read()
        state = load(pool, txn.handle)
        ASSERT state.stage == kVertexShaded

        lanes[] = load(state.vertex_lanes)
        triangles = [], raster_vtxout = []

        // Direct path: 2 triangles from 4/6 verts
        IF fill_solid / texture:
            ASSERT all verts inside homogeneous clip space

```

```

    FOR primitive IN {0, 1}:
        classify front-facing; check face cull
        BuildRasterTriangle(viewport transform)

// Indexed path
ELSE:
    lane_refs[] = load(state.vertex_lane_refs)
    FOR EACH segment:
        // check if any vertex needs clipping
        FOR EACH triangle (3 refs):
            vertices[3] = ReadClipVertex(lanes[ref])

            // Sutherland-Hodgman 6+8 plane clipping
            polygon = ClipTriangle(vertices)

            // Fan emission
            FOR fan = 2..polygon.size()-1:
                fan_tri = {poly[0], poly[fan-1], poly[fan]}
                front = ClassifyFrontFacing(fan_tri)
                culled = IsFaceCulled(state, front)
                IF clip_path AND culled: CONTINUE
                tri = BuildRasterTriangle(fan_tri, W, H)
                IF culled: tri.rasterizable = 0
                triangles.append(tri)

state.raster_triangles = store(triangles)
state.stage = kClipCullComplete
wait(cycles); store(state); output.write(txn)

```

13. Tiler Pseudocode

```

FUNCTION Tiler::Run():
    LOOP forever:
        txn = input_fifo.read()
        state = load(pool, txn.handle)
        ASSERT state.stage == kClipCullComplete

        triangles[] = load(state.raster_triangles)
        tiles = [], refs = []

        FOR y0 = 0, step 32:
            FOR x0 = 0, step 32:
                tile = {x0, y0, min(x0+32,W), min(y0+32,H)}
                tile.first_ref = refs.size()
                FOR EACH triangle:
                    IF rasterizable AND bbox_overlaps(triangle, tile):
                        refs.append({param_idx, submit_ordinal})
                tile.ref_count = refs.size() - tile.first_ref
                tiles.append(tile)

        state.tile_records = store(tiles)
        state.tile_primitive_refs = store(refs)
        state.stage = kTiled
        wait(cycles); store(state); output.write(txn)

```

14. ParameterBuffer Pseudocode

```

FUNCTION ParameterBuffer::Run():
    LOOP forever:
        txn = input_fifo.read()
        state = load(pool, txn.handle)
        ASSERT state.stage == kTiled

        triangles[] = load(state.raster_triangles)
        params = [], coeffs = []

        FOR EACH triangle:

```

```

IF !rasterizable:
    params.append(placeholder); CONTINUE

// Edge equations (24.8 fixed-point)
FOR edge IN {0,1,2}:
    Quantize coords, compute a/b/c, top-left rule

// Signed area + bounding box
signed_area = cross_product(quantized)
bbox = clamp(floor/ceil of min/max coords)

// Varying coefficients
IF has_varyings:
    coeffs.append(BuildPlane(tri, 1/w[])) // w plane
    FOR EACH varying component:
        IF FLAT: {0, 0, provoking_value}
        IF SMOOTH: BuildPlane(tri, v*1/w[])
        IF NOPERSP: BuildPlane(tri, v[])

params.append(param)

state.parameter_triangles = store(params)
state.parameter_coefficients = store(coeffs)
state.stage = kParameterBufferReady
wait(cycles); store(state); output.write(txn)

```